

On Spatial-Aware Community Search

Yixiang Fang¹, Zheng Wang, Reynold Cheng, *Member, IEEE*,
Xiaodong Li¹, Siqiang Luo¹, Jiafeng Hu¹, and Xiaojun Chen¹

Abstract—Communities are prevalent in social networks, knowledge graphs, and biological networks. Recently, the topic of community search (CS) has received plenty of attention. The CS problem aims to look for a dense subgraph that contains a query vertex. Existing CS solutions do not consider the spatial extent of a community. They can yield communities whose locations of vertices span large areas. In applications that facilitate setting social events (e.g., finding conference attendees to join a dinner), it is important to find groups of people who are physically close to each other, so it is desirable to have a *spatial-aware community* (or SAC), whose vertices are close structurally and spatially. Given a graph G and a query vertex q , we develop an exact solution to find the SAC containing q , but it cannot scale to large datasets, so we design three approximation algorithms. We further study the problem of continuous SAC search on a “dynamic spatial graph,” whose vertices’ locations change with time, and propose three fast solutions. We evaluate the solutions on both real and synthetic datasets, and the results show that SACs are better than communities returned by existing solutions. Moreover, our approximation solutions perform accurately and efficiently.

Index Terms—Community search, spatial graphs, geo-social networks, online queries

1 INTRODUCTION

WITH the emergence of geo-social networks, such as Twitter and Foursquare, the topic of geo-social networks has gained a lot of attention [1], [2], [3], [4]. In these networks, a user is often associated with location information (e.g., positions of her hometown and check-ins). These networks are collectively known as *spatial graphs*. Fig. 1 depicts a spatial graph with nine users in three cities Berlin, Paris, London, and each user has a specific location. The solid lines represent their social relationship, and the dashed lines denote their hometown locations.

In this paper, we study the problem of performing online community search (CS) on spatial graphs. Given a spatial graph G and a vertex $q \in G$, our goal is to find a subgraph of G , called a *spatial-aware community* (or SAC). Essentially, a community is a social unit of any size that shares common values, or that is situated in a close area [5], [6], [7]. An SAC is such a community with high *structure cohesiveness* and *spatial cohesiveness*. The structure cohesiveness mainly measures the social connections within the community, while the spatial cohesiveness focuses on the closeness among their geo-locations. Fig. 1 illustrates an SAC with three users {Tom, Jeff, Jim}, in which each user is linked with each other and all of them are in Paris.

Prior Works. The community retrieval (CR) methods can generally be classified into *community detection* (CD) and *community search*, as shown in Table 1. Earlier CD

methods [8], [9] mainly focus on link analysis without considering spatial features. Some recent studies [19] have shown that, in networks where vertices occupy positions in an Euclidian space, spatial constraints may have a strong effect on their relationship patterns, so some works [16], [17], [18] have considered the spatial features for CD. All these CD methods often detect all the communities from an entire graph using some predefined global criteria (e.g., modularity [20]), so their focus is beyond personalized CS. Also, their efficiency is inadequate for fast and online CR since they require to enumerate all the communities. To address these limitations, some works [10], [11], [12], [14], [15] focus on *online* CS, a query-dependent variant of CD, and they are able to find communities for a specific vertex. However, almost all these CS works focus on link analysis and do not consider the spatial features. In Fig. 1, for example, previous CS methods [10], [11] tend to put Jason and Tom, Jeff, Jim into the same community, although Jason is located in another city London. This community may not be very useful for some location-based services (e.g., setting up events). To alleviate this issue, in this paper we study SAC search which finds communities for a particular query vertex in an “online” manner. Our later experimental results on real datasets show that, the communities found by our methods are often in a much smaller areas than that of previous CS methods, i.e., the radii of the spatial circles covering communities found by [10] and [11] are 50 and 20 times larger than those of SAC search.

SAC Search. We now discuss how to measure the structure cohesiveness and spatial cohesiveness of an SAC. We adopt the commonly used metric *minimum degree* [10], [11], [13] to measure the structure cohesiveness. Note that in our method, the minimum degree metric can be easily replaced by other metrics like *k-truss* [14] and *k-clique* [12]. To measure the spatial cohesiveness, we consider the *spatial circle*, which contains all the community members. In particular, given a query vertex $q \in G$, our goal is to find an SAC containing q in the smallest *minimum covering circle* (or MCC) and all the vertices of the SAC satisfy the minimum degree

- Y. Fang, Z. Wang, R. Cheng, X. Li, S. Luo, and J. Hu are with the Department of Computer Science, The University of Hong Kong, Hong Kong. E-mail: {yxfang, zwang, ckcheng, xldli, sq Luo, jhu}@cs.hku.hk.
- X. Chen is with the College of Computer Science and Software, Shenzhen University, Shenzhen 518060, China. E-mail: xjchen@szu.edu.cn.

Manuscript received 14 Oct. 2017; revised 26 Apr. 2018; accepted 26 May 2018. Date of publication 8 June 2018; date of current version 5 Mar. 2019. (Corresponding author: Xiaojun Chen.)

Recommended for acceptance by L. B. Holder.

For information on obtaining reprints of this article, please send e-mail to: reprints@ieee.org, and reference the Digital Object Identifier below.

Digital Object Identifier no. 10.1109/TKDE.2018.2845414

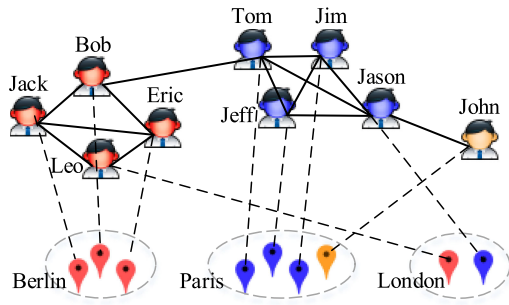


Fig. 1. A geo-social network.

TABLE 1
Works on Community Retrieval (CR)

Graph Type	Community Detection (CD)	Community Search (CS)
Non-spatial	[8], [9]	[10], [11], [12], [13], [14], [15]
Spatial	[16], [17], [18]	SAC search

metric. The main features of SAC search are summarized as follows.

- *Adaptability to location changes.* In geo-social networks (e.g., Brightkite and Foursquare), a user's location often changes frequently, due to its nature of mobility. As a result, users' spatially close communities change frequently as well. Let us consider two real examples in Brightkite, which once was a popular location-based social networking website. Fig. 2a shows a user's two SACs in two consecutive days, when she moves from place "A" to place "B" in US, in which each SAC is located in an MCC denoted by a circle. Note that all the members are different except the user herself. Fig. 2b shows another user's two SACs in three days, when she moves from place "C" to place "D". These real examples clearly show that a user's communities could evolve over time. In our later experiments, we find that for two SACs with time gap of six hours or more, the average Jaccard similarity of these two community member sets decreases by 25 percent.

Moreover, the link relationship also evolves over time. So the existing CD methods may easily lose the freshness and effectiveness after a short period of time. On the contrary, our SAC search can adapt to such dynamic easily, as it can answer queries in an "online" manner. Also, our methods do not rely on any offline computation, such as graph clustering or index structures.

- *Personalization.* SAC search allows a query user to find a community that exhibits both high structure cohesiveness and spatial cohesiveness. The parameter k , the minimum degree, allows the user to control the strength of link intensiveness. For example, SAC search can answer queries such as who are my nearby friends so that we can form a particular club? In contrast, existing CD methods [16], [17], [18] often use some global criteria (e.g., modularity), and consider the *static* community detection problem, where the graph is partitioned a-priori with no reference to the particular query vertices.

- *Online search.* Similar to other online CS methods, our method is able to find an SAC from a large spatial graph quickly once a query request arrives. However, existing CD methods for spatial graphs, are generally slower, as they are often designed for generating all the communities for an entire graph.

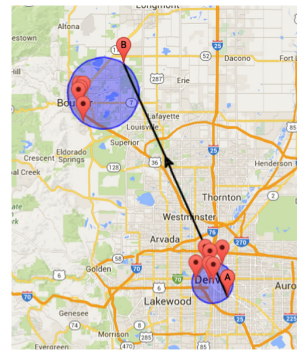
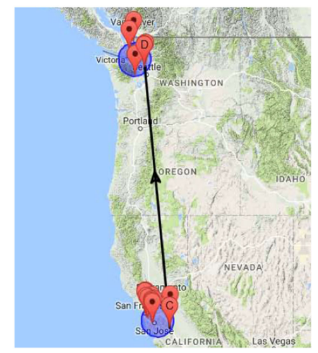
(a) user₁'s SACs(b) user₂'s SACs

Fig. 2. SACs in brightkite dataset.

Applications. We now discuss the applications of SAC search.

- *Event recommendation.* Emerging geo-social applications such as *Meetup* and *Meetin*¹ allow social network users to meet physically for various interesting purposes (e.g., party, dinner, and dating). For example, *Meetup* tracks its users' mobile phone locations, and suggests interesting location-based events to them [2]. Suppose that *Meetup* wishes to recommend an event to a user u . Then we can first find u 's SAC, whose members are physically close to u . Events proposed by u 's SAC member v can then be introduced to u , so that u can meet v if she is interested in v 's activity. Since u 's location changes constantly, u 's recommendation needs to be updated accordingly. Also, these applications often have to handle requests from a large number of online users efficiently. Our high-performance SAC search algorithms can therefore benefit these applications.

- *Social marketing.* As studied in [21], people with close social relationships tend to purchase in places that are also physically close. To boost sales figures, advertisement messages can be sent to the SACs of users who bought similar products before. For instance, if u has bought an item, the system can advertise this item to u 's SAC members.

- *Geo-social data analysis.* A common data analysis task is to study features about geographical regions. As discussed in [22], these features are often related to the people located there. For example, Silicon Valley can be characterized by "information technology" because many residents there are interested in this topic. Hence, by analyzing members of an SAC, it is possible to better understand the characteristics of a geographical area. As also discussed in [23] and Fig. 2, SAC search can be used to monitor and analyze the movement of communities. We can thus track the evolution and composition of u 's SAC as she moves.

Challenges and Contributions. The SAC search problem is very challenging, because the center and radius of the smallest MCC containing q are unknown. A basic exact approach takes $O(m \times n^3)$ time to answer a query, where n and m denote the numbers of vertices and edges in G . This is very costly, and is impractical for large spatial graphs with millions of vertices. So we turn to develop efficient approximation algorithms, which are able to find an SAC in an MCC of similar size with the smallest MCC. We first develop a basic approximation algorithm AppInc, which achieves an

1. Meetup: <https://www.meetup.com/> Meetin: <https://www.meetin.org/>

approximation of 2. Here, the approximation ratio is defined as the ratio of the radius of MCC returned over that of the optimal solution. Inspired by AppInc, we develop another approximation algorithm AppFast, which is faster and also has a more flexible approximation ratio, i.e., $2 + \epsilon_F$, where ϵ_F is an arbitrary small non-negative value. However, AppInc and AppFast cannot achieve even better accuracy with an approximation ratio less than 2. To tackle this issue, we further propose another approximation algorithm AppAcc with an approximation ratio of $1 + \epsilon_A$, where $0 < \epsilon_A < 1$. Overall, these approximation algorithms theoretically guarantee that, the radius of the MCC containing the SAC found has an arbitrary expected approximation ratio. Finally, inspired by the design of approximation algorithms, we develop an advanced exact algorithm Exact+, and our later experiments show that it is four orders of magnitude faster than the basic exact algorithm.

CSAC Search. In the geo-social networks above, the locations of vertices often change over time. For instance, a user may go to different restaurants in a day and perform “check-in” actions. As shown in Fig. 2 and our experiments in Section 6.2.2, a user’s SACs can change after a short period of time, due to the fact that every member of the SACs can move frequently. It is thus desirable to efficiently monitor the change of a user’s SACs. In this paper, we study the problem of *continuous SAC* (or CSAC) search, which aims to find the SACs of a query vertex q that can move as time goes by. The CSAC search can be applied to many social applications that need to know the latest communities of the users in order to provide the best recommendation. For example, in Meetup, when a user moves to a new place, it would be interesting to know her current location, and then provides event recommendation and product promotion services for her and members in her SAC in an instant manner.

A simple method to address the CSAC search problem is to invoke an SAC search algorithm upon every location change, which however is very inefficient, because users’ locations may change frequently, calling for more efficient algorithms. We observe that, although a user’s SACs may change frequently, they often share some vertices, and are spatially close or overlapped. Based on this intuition, we design three fast algorithms by extending SAC search algorithms, which are of theoretical accuracy guarantees.

We have implemented algorithms and performed extensive experiments on four real datasets and two synthetic datasets. We develop several metrics to measure the quality of a community with respect to the users’ social relationship and positions. Experimental results show that our solutions yield better communities than those produced by existing CS and CD algorithms. We have also studied the performance of different SAC search solutions. For moderate-size graphs, Exact+ is the best choice, as it achieves the highest quality with reasonable efficiency; for large graphs with millions of vertices or edges, AppFast and AppAcc are better options as they are much faster than Exact+. As shown in our experiments, our CSAC search algorithms are adaptive to frequent location changes, and faster than the baseline algorithms.

We review the related work in Section 2. We formulate the problem of SAC search in Section 3. Section 4 presents the algorithms for SAC search. In Section 5, we introduce the problem of CSAC search and develop three algorithms. We show the experimental results in Section 6 and conclude in Section 7.

2 RELATED WORK

Community Detection (CD). Classical CD solutions [8], [9], [24] often employ link-based analysis to obtain these communities. Some recent studies have used other advanced techniques, such as game-theoretic approach [25] and Laplacian centrality [26], to identify dense communities in large-scale complex networks. However, they do not consider the location information. Some recent works [16], [17], [18], [27] focus on identifying communities from spatially constrained graphs, whose vertices are associated with spatial coordinates [19]. For example, a geo-community [27] is like a community which is a graph of intensely connected vertices being loosely connected with others, but it is more compact in space. Guo et al. [16] proposed the average linkage measure for clustering objects in spatially constrained graphs. Expert et al. [17] uncovered communities from spatial graphs based on modularity maximization. In [18], Chen et al. used modularity maximization for CD on spatially constrained networks. We will compare it with our methods in experiments. The differences of CD algorithms and our SAC search are three-fold. First, CD algorithms are generally costly, as they often detect all the communities from an entire network. Second, it is not clear how they can be adapted for online CR. Third, as pointed out by [20], modularity based methods [17], [18] often fail to resolve small-size communities, even when they are well defined.

Community Search (CS). In recent years, there is another related but different problem of CD, called *community search*, which aims to obtain communities in an “online” manner, based on a query request. For example, given a vertex q , several existing works [10], [11], [12], [13], [14], [28] have proposed effective algorithms to obtain the most likely community that contains q . The *minimum degree* metric is often used to measure the structure cohesiveness of a community [10], [11]. In [10], Sozio et al. proposed the first algorithm Global to find the k -core containing q . In [11], Cui et al. proposed a more efficient algorithm Local, which uses local expansion techniques to boost the query performance. We will compare these two solutions in our experiments. In addition, some recent works [13], [15] also use the minimum degree metric to search communities from attributed graphs. Other well known structure cohesiveness, including k -clique [12], k -truss [14] and connectivity [29], have also been considered for online community search. But these works assume non-spatial graphs, and overlook the locations of vertices. Thus, it is desirable to investigate CS over spatial graphs.

In addition, graphs are often evolving in practice, which implies that the communities returned by CS solutions can become stale after a short period of time. Thus, more effective and efficient CS solutions on dynamic graphs are needed. However, designing such solutions is challenging due to the following reasons. First, network structures change dynamically. For example, users can join Facebook and make new friends frequently. Second, the attributes of vertices also update frequently. For instance, the locations of Facebook users may change quickly due to their nature of mobility. As a result, to search and track communities in such dynamic scenarios, we have to consider the dynamics from both network structures and attributes. Moreover, as reviewed below, although there are some existing studies on continuous queries, none of them has studied how to search communities

TABLE 2
Notations and Meanings

Notation	Meaning
$G(V, E)$	a graph with vertex set V and edge set E
n, m	the sizes of sets V and E respectively
$G[S]$	a subgraph of G induced by vertex set S
$nb(v)$	the neighbor set of vertex v in G
$deg_G(v)$	the degree of vertex v in G
$G' \subseteq G$	G' is a subgraph of G
$O(o, r)$	a spatial circle with center o and radius r
$ u, v $	the Euclidean distance from vertex u to vertex v
Ψ	Results of Exact and Exact+ (optimal solution)
Ψ'	Results of CExact+ (optimal solution)
Φ, Λ, Γ	Results of AppInc, AppFast, AppAcc respectively
Λ', Γ'	Results of CAppFast and CAppAcc
r_{opt}	The radius of the MCC covering Ψ
γ	The radius of MCC covering Φ
δ	The radius of MCC (centered at q) covering Φ
T	The quadtree used in AppAcc algorithm

continuously from dynamic spatial graphs. To fill this gap, in this paper we study continuous SAC search on spatial graphs.

Continuous Query (CQ). In recent years, the topic of CQ has received plenty of research interest, and many studies [30], [31], [32] focus on queries of spatial databases. In [30], the continuous queries over data streams are studied. Iwerks et al. [31] examine the continuous k -NN query. Wu et al. [32] study the continuous top- k spatial keyword query for a moving object. However, these studies focus on the spatial features [33], [34] of objects, and do not take the social network and communities into consideration. In recent studies [35], [36], the problem of maintaining k -core on dynamic graphs, where edges can be inserted and deleted dynamically, has been extensively studied. However, they do not consider the spatial features of objects. In summary, none of the existing works solves the problem of continuous SAC search.

3 PROBLEM DEFINITION

Data Model. We consider a geo-social network graph $G(V, E)$, which is an undirected graph with vertex set V and edge set E , where vertices represent entities and edges denote their relationships. For each vertex $v \in V$, it has a tuple (id, loc) , where id is its ID and $loc = (x, y)$ is its spatial positions along x - and y -axis in a two-dimensional space. Let n and m be the corresponding sizes of V and E . We illustrate the data model using Example 1. Table 2 shows the notations used in this paper.

Example 1. Fig. 3 a depicts a spatial graph, where each solid black line between two vertices represents an edge.

Spatial-Aware Community (SAC). Conceptually, an SAC is a subgraph, G' , of the graph G satisfying: (1) *Connectivity*: G' is connected; (2) *Structure cohesiveness*: all the vertices in G' are linked intensively; and (3) *Spatial cohesiveness*: all the vertices in G' are spatially close to each other. Note that all the existing CS studies assume that a community should satisfy the connectivity and structure cohesiveness [10], [11], [12], [13], [14], [28].

Structure Cohesiveness. A well-accepted notion of structure cohesiveness is the *minimum degree* of all the vertices that appear in the community is at least k [10], [11], [13], [37], [38]. This is used in k -core and our SAC search. Let us discuss the k -core first.

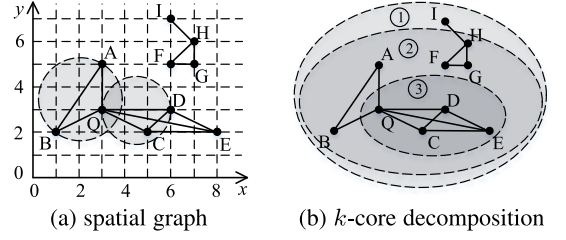


Fig. 3. An example of geo-social network.

Definition 1 (k -core [37], [38]). Given an integer k ($k \geq 0$), the k -core of G , denoted by H_k , is the largest subgraph of G , such that $\forall v \in H_k, deg_{H_k}(v) \geq k$.

We say that H_k has an order of k . The *core number* of a vertex $v \in V$ is then defined as the highest order of the k -core that contains v . A k -core has some properties [38]: (1) H_k contains at least $k + 1$ vertices; (2) H_k may not be a connected graph; (3) k -cores are nested, i.e., $H_{k+1} \subseteq H_k$; and (4) Computing the core numbers of all the vertices in a graph, also known as k -core decomposition, can be completed using a linear algorithm [38].

As a k -core may not be a connected subgraph, we denote its connected components by k -*cores*, which are usually the “communities” returned by k -*core* search algorithms [10], [11]. In Example 1, each k -core is covered by an ellipse as shown in Fig. 3b. Note that 2-core has two 2-*cores* with vertex sets $\{Q, A, B, C, D, E\}$ and $\{F, G, H\}$ respectively.

Remarks. Although we use the minimum degree in this paper, our proposed solutions can be easily adapted to other commonly-used structure cohesiveness metrics (e.g., k -truss [14] and k -clique [12]). For example, to use k -truss for any of our proposed algorithm, we can simply replace k -core by k -truss in the steps of verifying the structure cohesiveness of an SAC.

Spatial Cohesiveness. In this paper, to ensure high spatial cohesiveness, we require all the vertices of an SAC in a minimum covering circle (MCC) with the smallest radius. In the literature [39], [40], [41], [42], the notion of MCC has been widely adopted to achieve high spatial compactness for a set of spatial objects. The MCC and SAC search are defined as follows.

Definition 2 (MCC). Given a set of vertices S , the MCC of S is the spatial circle, which contains all the vertices in S with the smallest radius.

Problem 1 (SAC search). Given a graph G , a positive integer k and a vertex $q \in V$, return a subgraph $G_q \subseteq G$, and the following properties hold:

1. **Connectivity.** G_q is connected and contains q ;
2. **Structure cohesiveness.** $\forall v \in G_q, deg_{G_q}(v) \geq k$;
3. **Spatial cohesiveness.** The MCC of vertices in G_q satisfying Properties 1 and 2 has the minimum radius.

We call a subgraph satisfying properties 1 and 2 a *feasible* solution, and the subgraph satisfying all the three properties the *optimal* solution (denoted by Ψ). We denote the radius of the MCC containing Ψ by r_{opt} . Essentially, SAC search finds the SAC in an MCC with the smallest radius among all the feasible solutions. In Example 1, let $C_1 = \{Q, C, D\}$ and $C_2 = \{Q, A, B\}$. The two circles in Fig. 3a denote the MCCs of C_1 and C_2 respectively. Let $q = Q$ and $k = 2$. The optimal solution of this

query is $G[C_1]$, and $r_{opt} = 1.5$. Note that $G[C_2]$ and $G[C_1 \cup C_2]$ are feasible solutions. Additionally, although we only consider single query vertex in this paper, our proposed solutions can be easily extended to support multiple query vertices.

We also consider the θ -SAC search, which returns a community satisfying: properties 1 and 2 of SAC search, and all the vertices are in a spatial circle $O(q, \theta)$, where θ is an input parameter. This θ -SAC search is essentially a variant of Global [10] by introducing a parameter θ . Consider the graph in Example 1 with $q = Q$, $k = 2$ and $\theta = 3.1$. θ -SAC search will return $G[C_1 \cup C_2]$ as the community, as all of its vertices are in $O(Q, 3.1)$.

The θ -SAC query can be used when a user has some background knowledge (e.g., size of the region containing the SAC, and density of users in the region concerned). However, it can be difficult for a user of an application, such as Meetup, to specify an appropriate value of θ . As will be discussed in our experiments, the effectiveness of θ -SAC search is sensitive to θ . If θ is too small, no community can be found; if θ is too large, then the community is not spatially compact. A casual application user may then have to repeat the query with different θ values, before getting a satisfactory result. For the SAC search, the user does not need to specify θ ; instead, SAC search automatically suggests a community with tight structural and spatial cohesiveness. Thus, SAC search is more convenient to use than θ -SAC. In the above example, if $\theta < 2.2$, no community is found; if $\theta > 5.1$, $G[C_3]$ will be returned, where $C_3 = \{Q, A, B, C, D, E\}$. In fact, there are more spatially compact SACs (e.g., $G[C_1]$, $G[C_2]$ and $G[C_1 \cup C_2]$), among which the most compact one ($G[C_1]$) is returned by the SAC search.

4 SAC SEARCH ALGORITHMS

Most of our proposed algorithms follow a *two-step framework*: (1) find a community S of vertices, based on some CS algorithm e.g., Global [10], and (2) find a subset of S that satisfies both structure and spatial cohesiveness. Step (2) is computationally challenging; a simple way is to enumerate all the possible subsets of S , and then choose the one that satisfies the two criteria of SAC. In Example 1, when $q = Q$ and $k = 2$, $S = \{Q, A, B, C, D, E\}$; an SAC is then chosen from the $2^6 - 1 = 63$ subsets of S . This requires the examination of an exponential number of possible subsets of S in Step (2). In our experiments, the typical size of S ranges from 1K to 100K. As a result, the performance of SAC search can be seriously affected. Hence, we study polynomial-time SAC search algorithms for Step (2). Later we will also present the AppInc solution, which does not use Step (1).

We first present a basic exact algorithm Exact, which takes $O(m \times n^3)$ to answer a single query. This is very time-consuming for large graphs, so we turn to design more efficient approximation algorithms. Here, the approximation ratio is defined as the ratio of the radius of MCC returned over that of the optimal solution. Inspired by the approximation algorithms, we also design an advanced exact algorithm Exact+. Their approximation ratios and time complexities are summarized in Table 3, where ϵ_F and ϵ_A are parameters specified by the query user. The value $|F_1|$ is the number of “fixed vertices”, and $|F_1|$ is often much smaller than n . We will explain this parameter in Section 4.1.

AppInc is a 2-approximation algorithm, and it is much faster than Exact. Inspired by AppInc, we design another

TABLE 3
Overview of Algorithms for SAC Search

Algorithm	Approximation ratio	Time complexity
Exact	1	$O(m \times n^3)$
AppInc	2	$O(mn)$
AppFast	$2 + \epsilon_F$ ($\epsilon_F \geq 0$)	If $\epsilon_F > 0$, $O(m \cdot \min\{n, \log \frac{1}{\epsilon_F}\})$ If $\epsilon_F = 0$, $O(mn)$
AppAcc	$1 + \epsilon_A$ ($0 < \epsilon_A < 1$)	$O(\frac{m}{\epsilon_A^2} \times \min\{n, \log \frac{1}{\epsilon_A}\})$
Exact +	1	$O(\frac{m}{\epsilon_A} \cdot \min\{n, \log \frac{1}{\epsilon_A}\} + m F_1 ^3)$

($2 + \epsilon_F$)-approximation algorithm AppFast ($\epsilon_F \geq 0$), which is faster than AppInc. The limitation of AppInc and AppFast is that their approximation ratios are at least 2. To achieve even lower approximation ratio, we further design another algorithm AppAcc, whose approximation ratio is $(1 + \epsilon_A)$, where $0 < \epsilon_A < 1$ is a value specified by the query user. It is slightly slower than AppFast, as it spends more effort on finding more accurate solutions. Overall, these approximation algorithms guarantee that the radius of the MCC of the community has an arbitrary approximation ratio.

All algorithms except AppInc follow the two-step framework. Note that Step (1) of the two-step framework is not necessary for AppInc, since it works in an incremental manner. In addition, we can observe that, there is a trade-off between the quality of results and efficiency, i.e., algorithms with lower approximation ratios tend to have higher complexities. Our later experiments show that, for moderate-size graphs, Exact+ achieves not only the highest quality results, but also reasonable efficiency. While for large graphs with millions of vertices, AppFast and AppAcc should be better choices as they are much faster than Exact+.

4.1 The Basic Exact Algorithm

As mentioned before, a k -core contains at least $k + 1$ vertices. When $k = 1$, we can simply return the subgraph, induced by q and its nearest neighbor, as the result. So we focus on the case $k \geq 2$ in this paper. We begin with a useful lemma about MCC, described in [40], which inspires the design of our algorithms.

Lemma 1 ([40]). *Given a set S ($|S| \geq 2$) of vertices, its MCC can be determined by at most three vertices in S which lie on the boundary of the circle. If it is determined by only two vertices, then the line segment connecting those two vertices must be a diameter of the circle. If it is determined by three vertices, then the triangle consisting of those three vertices is not obtuse.*

By Lemma 1, there are at least two or three vertices lying on the boundary of the MCC of the target SAC. We call vertices lying on the boundary of an MCC *fixed vertices*. So a straightforward method of SAC search can follow the two-step framework directly. It first finds the k -core containing q , which is the same as Global does, and then returns the subgraph achieving both the structure and spatial cohesiveness by enumerating all the combinations of three vertices in the k -core. We denote this method by Exact. The pseudocodes are presented in the full version of this paper [43]. In addition, we show another two useful lemmas. Note all the proofs of lemmas proposed in this paper are discussed in [43].

Lemma 2 ([42]). *The maximum distance between any pair of vertices, u and v in Ψ , is in the range $[\sqrt{3}r_{opt}, 2r_{opt}]$.*

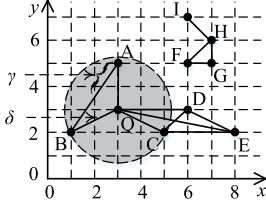


Fig. 4. Illustrating AppInc.

Lemma 3. The overall time complexity of *Exact* is $O(m \times n^3)$.

4.2 A 2-Approximation Algorithm

The major limitation of *Exact* is its high computational cost, which makes it impractical for large spatial graphs with millions of vertices. To alleviate this issue, we now develop more efficient approximation algorithms. We first present AppInc, which has an approximation ratio of 2. Our key observation is that, the optimal solution Ψ is usually very close to q . So we consider the smallest circle, denoted by $O(q, \delta)$, which is centered at q and contains a feasible solution, denoted by Φ . Let the radius of the MCC covering Φ be γ ($\gamma \leq \delta$). Note that, γ can be obtained by computing the MCC containing Φ by a linear algorithm [41]. Then, we have the following two interesting lemmas:

Lemma 4. $\frac{1}{2}\delta \leq r_{opt} \leq \gamma$.

Lemma 5. The radius of the MCC covering the feasible solution Φ has an approximation ratio of 2.

AppInc finds Φ in an incremental manner. Specifically, it considers vertices close to q one by one incrementally, and checks whether there exists a feasible solution when a new vertex is considered. It stops once a feasible solution has been found. The codes are detailed in [43]. We illustrate it by Example 2.

Example 2. In Example 1, let $q = Q$ and $k = 2$. AppInc first adds A to S and no SAC can be found. Then, it adds B to S , finds Φ with members set $\{Q, A, B\}$. So $\gamma = 1.803$ and $\delta = |Q, B| = 2.24$. The actual approximation ratio is $1.803/1.5 = 1.202$. Fig. 4 illustrate this.

Corollary 1. If q is the center of the MCC covering Ψ , AppInc finds the optimal solution, i.e., Φ equals to Ψ .

Corollary 2. The optimal solution Ψ is in $O(q, 2\gamma)$.

Lemma 6. The total time cost of AppInc is $O(mn)$.

4.3 A $(2+\epsilon_F)$ -Approximation Algorithm

Although AppInc is much faster than *Exact*, it is still inefficient for large graphs, since its time complexity is quadratic. In this section, we propose another fast approximation algorithm, called AppFast, which has a more flexible approximation ratio, i.e., $2 + \epsilon_F$, where ϵ_F is an arbitrary non-negative value.

Instead of finding the circle $O(q, \delta)$ in an incremental manner, AppFast approximates the radius δ by performing binary search. This is based on the observation that, the lower and upper bounds of δ , denoted by l and u , are stated by Eq. (1)

$$l = \max_{v \in KNN(q)} |q, v|, \quad u = \max_{v \in X} |q, v|, \quad (1)$$

where X is the list of vertices of the k -core containing q , and $KNN(q)$ contains the k nearest vertices in $X \cap nb(q)$ to q .

Authorized licensed use limited to: UNIVERSITY OF SHARJAH. Downloaded on April 16, 2025 at 03:47:54 UTC from IEEE Xplore. Restrictions apply.

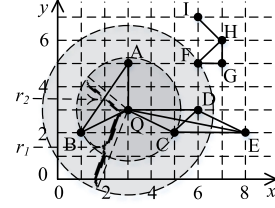
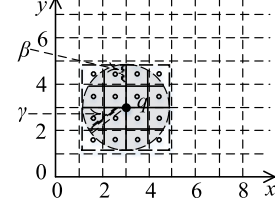


Fig. 5. Illustrating AppFast.

Fig. 6. Splitting $O(q, \gamma)$.

Hence, we can approximate the radius of the circle $O(q, \delta)$ by performing binary search within $[l, u]$ until the gap $|u - l|$ is less than α , a predefined threshold. More details of AppFast can be found in [43]. We illustrate AppFast by Example 3.

Example 3. In Fig. 5 ($q = Q$, $k = 2$, $\epsilon_F = 0.1$), AppFast first initializes $l = 2.24$, $u = 5.10$, and tries to find a feasible solution from $O(Q, r_1)$ and $O(Q, r_2)$, where $r_1 = 3.67$ and $r_2 = 2.24$. It stops after searching $O(Q, r_2)$, as $r_2 - l = 0$. Λ equals to Φ .

Lemma 7. In AppFast, the radius of the MCC covering Λ has an approximation ratio of $(2 + \epsilon_F)$, if α is set as $\frac{r \times \epsilon_F}{2 + \epsilon_F}$.

Corollary 3. The optimal solution Ψ is in $O(q, 2r_\Lambda)$, where r_Λ is the radius of the MCC containing Λ in AppFast.

Lemma 8. The total time cost of AppFast is $O(\min\{mn, m \log \frac{1}{\epsilon_F}\})$ if $\epsilon_F > 0$, or $O(mn)$ if $\epsilon_F = 0$.

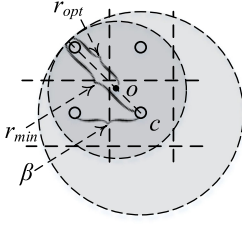
4.4 A $(1+\epsilon_A)$ -Approximation Algorithm

AppInc and AppFast guarantee that, the radius of the MCC of the returned SAC has an approximation ratio of 2 or more, but cannot achieve even better accuracy. To tackle this issue, we propose another algorithm, called AppAcc, which has an approximation ratio of $(1+\epsilon_A)$, where $0 < \epsilon_A < 1$. The main idea is based on a key observation from Lemma 4, stated by Corollary 4:

Corollary 4. The center point, o , of the MCC $O(o, r_{opt})$ covering Ψ is in the circle $O(q, \gamma)$.

Although point o is in $O(q, \gamma)$, it is still not easy to locate it exactly, since the number of its possible positions to be explored can be infinite. Instead of locating it exactly, we try to find an approximated "center", which is very close to o . In specific, we split the square containing the circle $O(q, \gamma)$ into equal-sized cells, and the size of each cell is $\beta \times \beta$ (we will explain how to set a proper value of β later). We call the center point of each cell an *anchor point*. By Corollary 4, we can conclude that o must be in one specific cell. Then we can approximate o using the anchor point of this cell, denoted by c , which is also its nearest anchor point, since their distance $|o, c|$ is at most $\frac{\sqrt{2}}{2}\beta$.

Example 4. In Fig. 6, each small circle point in $O(q, \gamma)$ represents an anchor point. In Fig. 7, c is the nearest anchor point of o . It is easy to observe that $|o, c| \leq \frac{\sqrt{2}}{2}\beta$.

Fig. 7. Illustrating r_{min} .

We consider the circle $O(c, r_{min})$, where r_{min} is the minimum radius such that it contains a feasible solution, which is denoted by Γ . The value of r_{min} is bounded by Lemma 9.

Lemma 9. $r_{min} \leq r_{opt} + \frac{\sqrt{2}}{2} \beta$.

By Lemma 9, we have $\frac{r_{min}}{r_{opt}} \leq 1 + \frac{\sqrt{2}\beta}{2r_{opt}} \leq 1 + \frac{\sqrt{2}\beta}{\delta}$. Thus, we can approximate Ψ using Γ , and the approximation ratio is $(1 + \epsilon_A)$, if we let $\frac{\sqrt{2}\beta}{\delta} \leq \epsilon_A$ ($0 < \epsilon_A < 1$).

To find $O(c, r_{min})$, the basic method is that, for each anchor point p , we use AppFast to find the circle, which is centered at p and contains a feasible solution, and then return the minimum circle. However, the number of anchor points is $(\frac{2\gamma}{\beta})^2$, and each takes $O(mn)$ to find a feasible solution in the worst case. So this is very time-consuming, if β (ϵ_A) is very small. To further improve the efficiency, we develop some optimization techniques.

Specifically, we assume that all the anchor points are organized into a region quadtree [44], where the root node² is a square, centered at q with width 2γ . By decomposing this square into four equal-sized quadrants, we obtain its four child nodes. The child nodes of them are built in the same manner recursively, until the width of the leaf node is in $(\beta/2, \beta]$. Note that the center of each leaf node corresponds to an anchor point.

To find $O(c, r_{min})$, we traverse the quadtree level by level in a top-down manner. Let r_{cur} , initialized as γ , record the smallest radius of an MCC containing a feasible solution. For each node, we first obtain the center p of its square, and then use the binary search technique introduced in AppFast to approximate the smallest radius r_p , such that $O(p, r_p)$ contains a feasible solution. During the traversal, for each node, to check whether it can be pruned, we propose three effective pruning criteria:

- **Pruning1:** Consider a node (with center p), which intersects at the boundary of $O(q, r_{cur})$. Then we have $|p, q| \leq r_{cur} + \frac{\sqrt{2}}{2} \beta$. Thus, if the distance from the center of this node to q is larger than $r_{cur} + \frac{\sqrt{2}}{2} \beta$, its sub-trees can be pruned.
- **Pruning2:** If $O(p, r)$ does not contain a feasible solution and $r > r_{cur} + \frac{\sqrt{2}}{2} \beta$, then its sub-trees can be pruned.
- **Pruning3:** Let Γ_i be the SAC found by the binary search on the i th anchor point, and M_i be the MCC covering Γ_i . If there are more than one anchor points whose corresponding SACs are the same, say Γ_a , then we can only keep one anchor point b whose distance to the center of M_a is the minimum.

In addition, we propose a lemma, which may stop the process of searching the quadtree earlier, as follows.

Lemma 10. In AppAcc, if $r_{cur} \leq \frac{1}{2}\delta(1 + \epsilon_A)$, then we can stop searching the quadtree and return the SAC with radius r_{cur} .

Based on the analysis above, we design AppAcc. It first performs AppFast, and obtains the k -core in $O(q, 2\gamma)$. Then, it explores the quadtree level by level with pruning in a top-down manner and tries to find feasible solutions using binary search as that of AppFast, until the cell size is at most $\beta \times \beta$. Finally, it returns the SAC (denoted by Γ) in the smallest MCC centered at a specific anchor point. More details are described in [45].

Lemma 11. If we set $\alpha' \leq \frac{1}{4}\delta\epsilon_A$ and $\beta = \frac{\delta\epsilon_A}{\sqrt{2}(2+\epsilon_A)}$ in AppAcc, where α' is the threshold of the gap between the upper and lower bounds of the binary search, and $0 < \epsilon_A < 1$, the radius of the MCC covering Γ has an approximation ratio of $(1 + \epsilon_A)$.

Lemma 12. AppAcc takes $O(m(\frac{1}{\epsilon_A})^2 \times \min\{n, \log \frac{1}{\epsilon_A}\})$ time.

4.5 The Advanced Exact Algorithm

The design of previous algorithms provide us many useful insights for developing more advanced exact algorithms. For example, Corollary 2 states that, the optimal solution Ψ is in $O(q, 2\gamma)$. This implies that, we can first run AppInc, then only enumerate the vertex triples for vertices in $O(q, 2\gamma)$, which is a subset of V . Similarly, we can find Ψ by Corollary 3 based on AppFast. Although these methods could be faster than Exact, they are still far from perfect, because the number of potential fixed vertices in $O(q, 2\gamma)$ may still be very large. In this section, we propose a very efficient exact algorithm based on AppAcc, called Exact+, which largely reduces the number of potential fixed vertices, and thus improves the efficiency significantly.

Recall that, AppAcc approximates the center, o , of the MCC covering Ψ by its nearest anchor point c , and $|o, c| \leq \frac{\sqrt{2}}{2} \beta$. Also, r_{opt} is well approximated, i.e., $\frac{r_{\Gamma}}{r_{opt}} \leq 1 + \epsilon_A$, which implies that,

$$\frac{r_{\Gamma}}{1 + \epsilon_A} \leq r_{opt} \leq r_{\Gamma}, \quad (2)$$

where $0 < \epsilon_A < 1$. So the value of r_{opt} is in a small interval, especially if ϵ_A is small.

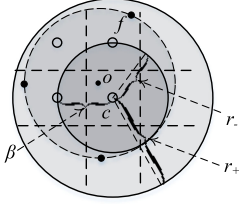
Besides, for any fixed vertex, f , of the MCC of Ψ , its distance to o (i.e., $|f, o|$) is exactly r_{opt} . By triangle inequality, we have

$$|f, c| \leq |f, o| + |o, c| \leq r_{\Gamma} + \frac{\sqrt{2}}{2} \beta, \quad (3)$$

$$|f, c| \geq |f, o| - |o, c| \geq \frac{r_{\Gamma}}{1 + \epsilon_A} - \frac{\sqrt{2}}{2} \beta. \quad (4)$$

Let us denote the rightmost items of above two inequalities by r_+ and r_- respectively. Then, we conclude that, for any fixed vertex f , its distance to c is in the range $[r_-, r_+]$. If ϵ_A is very small, the gap between r_+ and r_- , i.e., $r_+ - r_- = r_{\Gamma}(1 - \frac{1}{1+\epsilon_A}) + \sqrt{2}\beta$, is also very small, which implies that the locations of the fixed vertices are in a very narrow annular region. Hence, a large number of vertices out of this annular region, which are not fixed vertices, can be pruned safely. We illustrate this in Fig. 8, in which the annular region is the area in $O(c, r_+)$, but not in $O(c, r_-)$. Based on the analysis above, we design Exact+, which first runs AppAcc, then prunes fixed vertices using annular regions, and finally computes the target SAC by enumerating all the

2. To avoid ambiguity, we use word “node” for tree nodes.

Fig. 8. Exact+: r_- , r_+ .

combinations of three fixed vertices. The detailed pseudocodes of Exact+ are presented in [43].

Lemma 13. *The overall time cost of Exact+ is $O(m(\frac{1}{\epsilon_A})^2 \times \min\{n, \log \frac{1}{\epsilon_A}\} + m|F_1|^3)$, where ϵ_A is the parameter of AppAcc and $|F_1|$ ($|F_1| \ll n$) is the number of fixed vertices left after pruning by annular regions.*

5 CSAC SEARCH

In this section, we first formally define problem of CSAC search and then introduce the proposed algorithms for CSAC search.

5.1 Problem Definition

Data Model. We consider a *dynamic spatial graph* $G(V, E)$, which is an undirected graph with vertex set V and edge set E , where vertices represent entities and edges denote their relationships. Besides, each vertex v has a triple (id, loc, t) , where id is v 's ID and loc is v 's location at the timestamp t , and its location loc changes as the time t goes on.

Given a query vertex, the goal of CSAC search is to dynamically track her SACs in a future time interval. We formulate the CSAC search problem as follows and illustrate it by Example 5.

Problem 2 (CSAC search). *Given a dynamic spatial graph G , a positive integer k , a future timestamp t_e , and a vertex $q \in V$, find a list $G_q(t)$ of SACs such that*

$$G_q(t) = \begin{cases} G_{q,1}, & t \in [t_s, t_1) \\ G_{q,2}, & t \in [t_1, t_2) \\ \dots, & t \in [\dots, \dots) \\ G_{q,l}, & t \in [t_{l-1}, t_e] \end{cases}, \quad (5)$$

where t_s is the timestamp of starting the query, t_i ($1 \leq i \leq l-1$) is a future timestamp, l is the total number of SACs, $G_{q,i}$ is the i th SAC, and $G_{q,i}$ is different with $G_{q,i+1}$.

Example 5. Fig. 9 shows a dynamic spatial graph during the time period $[t_0, t_2]$, where t_0 is the current timestamp. Note that initially the graph is the same as that in Fig. 3a. Let $q = Q$, $k = 2$ and $t_e = t_2$, we can observe that: when the time $t = t_0$, the SAC of Q is $G[C_1]$; at a later time $t = t_1$, the SAC of Q changes to $G[C_2]$; and then $t = t_2$, the SAC of Q changes to $G[C_4]$, $C_4 = \{Q, C, E\}$. Thus, the result of this query is

$$G_Q(t) = \begin{cases} G[C_1], & t \in [t_0, t_1) \\ G[C_2], & t \in [t_1, t_2) \\ G[C_4], & t \in [t_2, t_2] \end{cases}. \quad (6)$$

The dynamics of an SAC are two-fold: (1) its member set may change since its members may move out of the MCC of the SAC, and other members which are not from the SAC may be close to q after their movement; and (2) its location

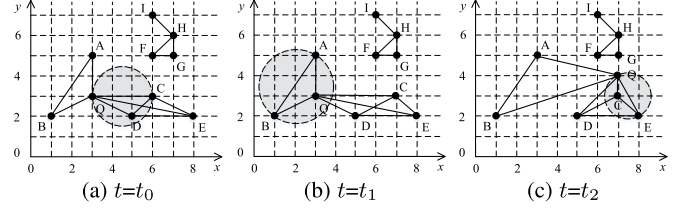


Fig. 9. An example of CSAC search.

may change, even when its member set remains unchanged after its members' movement. A simple method to address the CSAC search problem is to invoke an SAC search algorithm upon every location change. This method, however, is very inefficient due to the users' nature of high mobility. Thus, it is desirable to develop fast algorithms.

We observe that, although a user's SACs change frequently, they often share some vertices, and are spatially close or overlapped. Based on this intuition, we design three fast algorithms, i.e., CAppFast, CAppAcc, and CExact+, by extending AppFast, AppAcc and Exact+ respectively. These algorithms generally follow the same framework, and guarantee that the returned SAC achieves an arbitrary small approximation ratio. Note that, we do not extend AppInc and Exact because they are slower than AppFast and Exact+ respectively [45].

CAppFast is a $(2+\epsilon_F)$ -approximation continuous algorithm, where $\epsilon_F \geq 0$, and its theoretical approximation ratios is at least 2. The CAppAcc algorithm is extended from AppAcc, which achieves an approximation ratio of $1 + \epsilon_A$, where $0 < \epsilon_A < 1$ is a value specified by the query user. It is slightly slower than CAppFast, as it spends more effort on finding more accurate solutions. Similarly, we design a fast exact continuous algorithm CExact+ based on Exact+. In the following, we first discuss the framework and then introduce the algorithms one by one.

Remark. In this paper, we mainly focus on the dynamic updates from vertices' locations; for the changes of edges (relationship) of vertices, there are many existing algorithms [35], [36] for maintaining the k -core dynamically, which can be easily incorporated into our framework, and thus we skip them.

5.2 The Framework of CSAC Algorithms

In the framework, we consider the updates one by one and allow any of the extended algorithms to be incorporated. We denote a specific algorithm for static scenario by Algo (one of AppFast, AppAcc, and Exact+), and also denote its extended version for dynamic graph by CALgo (one of CAppFast, CAppAcc, and CExact+ respectively). We let the vertex that generates the update be p . After p 's movement, we denote it by p' . In other words, p and p' are of the same id , but different locations and timestamps. Similarly, we let the SACs and radiuses of MCCs covering them before and after p 's movement be Δ and r , Δ' and r' respectively. In addition, for each instance of CALgo, in the framework we maintain additional variables, which will be updated during running CALgo. We will discuss them later.

In the framework, we sequentially consider the moving records one by one, and update q 's SACs based on a key observation that, if a moving vertex is too far from q 's SAC, then we may skip it directly as its movement may not

change the SAC; otherwise, we need to call `CALgo` to update the SAC. The main technical challenge is how to design a fast algorithm `CALgo`, so that we can avoid invoking `Algo` and reduce the cost of updating the SAC. We will detail this in the following sections.

Algorithm 1. Framework of CSAC Algorithms

```

1: function CSAC( $G, q, k, \epsilon, t_e, \text{Algo}, \text{CALgo}$ )
2:   find the vertex list  $X$  of the  $k$ -core containing  $q$ ;
3:   sort vertices of  $X$  and store them in  $\text{PriQueue}$ ;
4:   initialize  $t_s$  as the current timestamp;
5:   while current time  $t \leq t_e$  and has an update  $(p, p')$  do
6:     if  $p.t < t_s$  and  $p'.t \geq t_s$  then
7:       call Algo( $G, q, k, \epsilon$ ) to initialize  $\Delta$  and  $r$ ;
8:       output  $\Delta$ ;
9:     else
10:      if  $p \notin X$  or  $(|p', q| > 2r \text{ and } |p, q| > 2r)$  then continue;
11:      call CALgo( $G, p, p', q, k, \epsilon$ ) to get  $\Delta'$  and  $r'$ ;
12:      if  $\Delta' \neq \Delta$  then
13:        output  $\Delta$ , and  $[t_s, t)$  for  $\Delta'$ ;
14:         $\Delta \leftarrow \Delta', r \leftarrow r', t_s \leftarrow t$ ;
15:      update  $p$  by  $p'$ ; // update the graph
16:      output  $[t_s, t_e]$  for  $\Delta$ ;
```

Algorithm 1 presents the framework. The input is a dynamic spatial graph G , a vertex q , an integer k , a non-negative value ϵ , a future timestamp t_e , and two algorithms. During the dynamic location change process, a list of different SACs of q are outputted. We first find the k -core of q , and put its vertices into X and PriQueue , which gives high priorities for vertices near to q (lines 2-3). Then, we initialize a variable t_s as the current timestamp (line 4). Next, in the while loop (lines 5-15), we monitor all the updates and handle them one by one. If the update (from p to p') is the first one, then we initialize Δ and r by calling `Algo` (lines 6-8); otherwise, we decide whether to update Δ by `CALgo` using Lemma 14. In case that we need to update Δ and r , we run `CALgo` (lines 11-14). Finally, we update the graph G (line 15). Note that the duration of each returned SAC is also reported (lines 13, 16).

Lemma 14. If $p \in X$, $|p, q| > 2r$, and $|p', q| > 2r$, we have $\Delta' = \Delta$, which means the movement of p does not change the SAC.

5.3 The CAppFast Algorithm

Similar to the idea of `AppFast`, we perform binary search to find the target SAC in `CAppFast`. The key step of `CAppFast` is to compute the tight upper and lower bounds, i.e., l' and u' , of the binary search, so that we can update SAC efficiently. In the framework, we maintain an additional variable $\hat{p}$, called *mark vertex*, which is the farthest vertex to q in the returned SAC.

To compute Λ' , the updated SAC after p 's movement, we consider three cases. (1) If $p.id = q.id$, which implies that q moves, we have to run `AppFast` again. (2) If $p.id \neq q.id$ and $p = \hat{p}$, which implies the mark vertex $\hat{p}$ moves, then we have

- If $|p', q| < r_\Lambda$, then $l' = |p', q|$, $u' = |q, \text{PriQueue.tail}|$.
- If $|p', q| \geq r_\Lambda$, then $l' = |q, \text{PriQueue.peak}|$, $u' = |p', q|$.
- (3) If $p.id \neq q.id$ and $p' \neq \hat{p}$, we have
 - $|p', q| < r_\Lambda$, then $l' = |p', q|$, $u' = r_\Lambda$.
 - $|p', q| \geq r_\Lambda$, then $l' = r_\Lambda$, $u' = |p', q|$.

Algorithm 2 presents `CAppFast`. If q moves, we compute the lower and upper bounds (lines 2-3); otherwise, we check whether p is $\hat{p}$ or not. If $\hat{p}$ moves, we

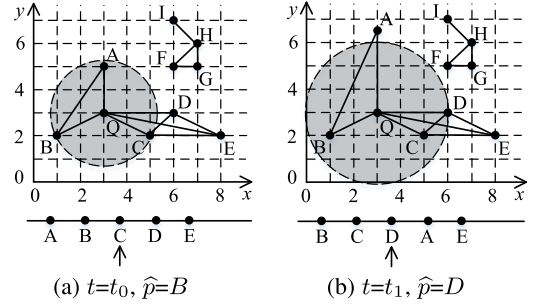


Fig. 10. Illustrating `CAppFast`.

we get the lower and upper bounds by the *PriQueue* (lines 4-8); otherwise, we reset the bounds (lines 9-11). After that, we perform the binary search to find the SAC (line 12), which is the same as lines 4-14 of `AppFast`. Finally, $\hat{p}$ is updated and Λ' is returned (lines 13-14). For the update of $\hat{p}$, we can simply reset it as the farthest vertex to q in the returned SAC according to the definition of mark vertex.

Algorithm 2. Query Algorithm: CAppFast

```

1: function CAppFast( $G, p, p', q, k, \epsilon_F$ )
2:   if  $p'.id = q.id$  then // case (1)
3:      $l' = \max_{p \in X \cap nb(q)} |q, v|$ ,  $u' = \max_{v \in \Lambda} |q, v|$ ;
4:   if  $p'.id = \hat{p}.id$  then // case (2)
5:     if  $|p', q| < r_\Lambda$  then
6:        $l' \leftarrow |p', q|$ ,  $u' \leftarrow |q, \text{PriQueue.tail}|$ ;
7:     else
8:        $l' \leftarrow |q, \text{PriQueue.peak}|$ ,  $u' \leftarrow |p', q|$ ;
9:   else // case (3)
10:    if  $|p', q| < r_\Lambda$  then  $l' \leftarrow |p', q|$ ,  $u' \leftarrow r_\Lambda$ ;
11:    else  $l' \leftarrow r_\Lambda$ ,  $u' \leftarrow |p', q|$ ;
12:     $\text{BiSearch}(l', u')$ ; // lines 4-14 in AppFast
13:    update the additional variable  $\hat{p}$ ;
14:    return  $\Lambda'$ ;
```

Next, we elaborate `CAppFast` via Example 6.

Example 6. Consider the graph in Fig. 10 where t_0 is the current timestamp. Let $q = Q$, $k = 2$, and $t_e = t_1$. We first sort the vertices in X : A, B, C, D, E , and put them in PriQueue . Then, we follow the framework and run `AppFast` to initialize Λ which contains Q, A , and B , $r_\Lambda = |B, Q|$, and $\hat{p} = B$. Next, A moves out of $O(q, |Q, B|)$ at $t = t_2$, the queue changes to B, C, D, A, E . The upper and lower bounds of the binary search are $l' = r_\Lambda$, $u' = |A, Q|$. After the binary search, we update $\hat{p} = D$ and $r_\Lambda = |D, Q|$. Note that the SACs are circled.

5.4 The CAppAcc Algorithm

Recall that in `AppAcc`, to find a $(1 + \epsilon_A)$ -approximation solution Γ , we explore a quadtree centered at q . After p 's movement, to find an updated $(1 + \epsilon_A)$ solution Γ' efficiently, we reuse the previous quadtree so that the cost of finding Γ' is minimized.

In `CAppAcc`, we need to maintain several additional variables, which are the quadtree $\mathcal{T}$, γ , r_Γ , and δ . These variables are updated after running `CAppAcc`. We denote the corresponding variables of $\mathcal{T}$, γ , r_Γ , and δ after p 's movement by $\mathcal{T}'$, γ' , r_Γ' , and δ' respectively. To simplify the presentation, we consider two cases in `CAppAcc`, where the first case is

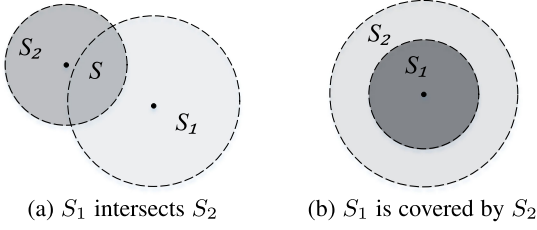


Fig. 11. Illustrating CAppAcc.

that q moves while the second case is that q does not move. We discuss them as follows.

5.4.1 The First Case ($q.id = p.id$)

Since the query vertex q moves, the center of quadtree changes. Considering whether the areas covering T and T' intersect or not, we have the following two subcases:

(1) If $|p', q| \geq \gamma' + \gamma$, then the areas of T and T' are separated or tangent, and we need to rerun AppAcc to obtain Γ' .

(2) If $|p', q| < \gamma' + \gamma$, then the areas of T and T' are intersected with each other. We illustrate this in Fig. 11a, where S_1 is the area of T , S_2 is the area of T' , and S the intersection area. Let $r_\Gamma[S]$ denote the minimum radius of an MCC, whose corresponding anchor is in S and contains a feasible solution. Then, we need to find a feasible solution and the radius of MCC covering it is $r_{\Gamma'} = \min\{r_\Gamma[S], r_{\Gamma'}[S_2 - S]\}$. For the area S , we can search the community by directly using T ; while for $S_2 - S$, we need to search the community using T' .

5.4.2 The Second Case ($q.id \neq p.id$)

In this case, the center of the quadtree does not change after q 's movement (i.e., it is at q), but its size may change. We proceed to discuss the algorithm by considering three subcases:

(1) If $\gamma = \gamma'$ and $\delta = \delta'$, then we have $\Gamma' = \Gamma$. The reason is that, by Lemma 4, we have $1/2\delta' \leq r_{opt}' \leq \gamma'$. This implies that if $\frac{r_\Gamma}{r_{opt}'} \leq \frac{r_\Gamma}{1/2\delta'} = \frac{r_\Gamma}{1/2\delta} \leq 1 + \epsilon_A$, we have $\frac{r_\Gamma}{r_{opt}'} \leq 1 + \epsilon_A$.

(2) If $\gamma' \leq \gamma$ and $\delta \neq \delta'$, the movement of p is likely to change the fixed vertexes of the MCC covering the optimal solution. So we need to search anchor points in T' . Since $r_{opt} \leq r_\Gamma \leq \gamma'$, we can narrow the area containing T' as $O(q, r_\Gamma)$ and find Γ' from T' .

(3) If $\gamma' > \gamma$, then the area of T is fully contained by the area of T' . We explain this in Fig. 11b, where S_1, S_2 are the areas of T and T' respectively. It is easy to observe that, $r_{\Gamma'} = \min\{r_\Gamma[S_1], r_{\Gamma'}[S_2 - S_1]\}$. In addition, since q does not move, the center of the quadtree does not change, we only need to update T , if S_2 is larger than S_1 after running CAppAcc.

5.4.3 The Overall Algorithm of CAppAcc

We present the overall steps of CAppAcc in Algorithm 3. It first computes γ' using AppFast and then considers two cases for finding Γ' separately. The first case is that q moves (lines 3-10), and the second case is that q does not move (lines 11-21). The steps for each case exactly follow our previous discussion. After obtaining Γ' , we update all the additional variables introduced in the framework (line 22), and return Γ' (line 23).

5.5 The CExact+ Algorithm

Recall that in Exact+, we first get the anchor points using AppAcc and then use the annular region to prune a large number of fixed vertices. In CExact+, we let r_{opt}' be the

radius of MCC covering the optimal solution Ψ' after p 's movement. We follow the framework, and maintain two additional variables F_1 and R_1 , where F_1 is the set of the candidate fixed vertices and R_1 is the set of the fixed vertex for the latest optimal solution Ψ . Moreover, we propose Lemmas 15 and 16, which greatly reduce the size of the annular region and improve the query efficiency.

Algorithm 3. Query Algorithm: CAppAcc

```

1: function CAppAcc( $G, p, p', q, k, \epsilon_A$ )
2:   compute  $\gamma'$  using AppFast;
3:   if  $p'.id = q.id$  then // The first case
4:     if  $|p', q| \geq \gamma' + \gamma$  then
5:       run AppAcc( $G, q, k, \epsilon_A$ ) to get  $\Gamma'$ ;
6:     else
7:       obtain the intersection area  $S$ ;
8:       compute  $r_S$  using  $T$  in area  $S$ ;
9:       compute  $r_{S'}$  in  $S_2 - S$ ;
10:      find  $\Gamma'$  such that  $r_{\Gamma'} = \min(r_S, r_{S'})$ ;
11:   else // The second case
12:     if  $\gamma' = \gamma$  and  $\delta' = \delta$  and  $\frac{2r_\Gamma}{\delta} \leq 1 + \epsilon_A$  then
13:        $r_{\Gamma'} \leftarrow r_\Gamma, \Gamma' \leftarrow \Gamma$ ;
14:     else
15:       if  $\gamma' \leq \gamma$  then
16:         find  $\Gamma'$  in  $O(q, r_\Gamma)$  using  $T$ ;
17:       else
18:         obtain the intersection area  $S$ ;
19:         compute  $r_S$  using  $T$  in area  $S$ ;
20:         compute  $r_{S'}$  in  $S_2 - S$ ;
21:         find  $\Gamma'$  such that  $r_{\Gamma'} = \min(r_S, r_{S'})$ ;
22:     update additional variables  $T, \gamma, r_\Gamma$ , and  $\delta$ ;
23:   return  $\Gamma'$ ;

```

Recall that when computing Ψ in Exact+, we have introduced variables $\delta, r_\Gamma, \gamma, r_-, r_+$. We denote these corresponding variables when computing Ψ' by $\delta', r_{\Gamma'}, \gamma', r_-', r_+' respectively. We now present two interesting lemmas.$

Algorithm 4. Query Algorithm: CExact+

```

1: function CExact+( $G, p, p', q, k, \epsilon_A$ )
2:   run CAppAcc( $G, p, p', q, k, \epsilon_A$ ) to get  $\Gamma'$ ;
3:   obtain  $r_-'$  and  $r_+' using Lemmas 15 and 16;
4:   if  $F_1 = F_1'$  and  $p' \notin F_1$  then return  $\Psi$ ; // case (a)
5:   if  $F_1 = F_1'$  and  $p' \in F_1$  and  $p' \notin F_1$  then // case (b)
6:      $\Psi' \leftarrow \text{enumFix}(p', F_2', F_3')$ ;
7:   else // case (c)
8:      $\Psi' \leftarrow \text{enumFix}(F_1', F_2', F_3')$ ;
9:   update additional variables  $F_1$  and  $R_1$ ;
10:  return  $\Psi'$ ;$ 
```

Lemma 15. The range of r_{opt}' is

$$\max\left\{\frac{\delta'}{2\gamma'} r_{opt}, \frac{r_{\Gamma'}}{1 + \epsilon_A}\right\} \leq r_{opt}' \leq \min\left\{\frac{2\gamma'}{\delta} r_{opt}, r_{\Gamma'}\right\}. \quad (7)$$

Lemma 16. If $\delta' = \delta$ and $\frac{2r_\Gamma}{\delta} - 1 \leq \epsilon_A \leq \frac{2\gamma'}{\delta} - 1$, then the range of r_{opt} can be further contracted as

$$\max\left\{\frac{\delta'}{2\gamma'} r_{opt}, \frac{r_{\Gamma'}}{1 + \epsilon_A}\right\} \leq r_{opt}' \leq r_{\Gamma'}. \quad (8)$$

TABLE 4
Datasets Used in Our Experiments

Type	Name	Vertices	Edges	$\hat{d}$	BC	CC
Real	Brightkite	51,406	197,167	7.67	7.34E-5	0.1795
	Gowalla	107,092	456,830	8.53	3.40E-5	0.2487
	Flickr	214,698	2,096,306	19.5	1.65E-5	0.1113
	Foursquare	2,127,093	8,640,352	8.12	1.68E-6	0.1044
Synthetic	Syn1	30,000	300,000	20	1.26E-4	0.0077
	Syn2	400,000	4,000,000	20	1.10E-5	0.0006

Similar to Exact+, we can compute r_-' and r_+' . After that, we use the annular region to find the set F_1' of candidate fixed vertices and the set R_1' of fixed vertices. To improve the efficiency of query, we further consider three cases by proposing Lemma 17.

Lemma 17. *Let F_1' , F_2' , and F_3' be the corresponding variables of F_1 , F_2 and F_3 in Exact+ after p 's movement. We have 3 cases: (a) If $F_1 = F_1'$ and $p' \notin F_1$, we can conclude that the movement of p will not affect F_1 . In other words, we have $\Psi' = \Psi$. (b) If $F_1 = F_1'$ and $p' \in F_1$, and $p' \notin R_1$, then we only need to enumerate all the combinations of three vertices, where one is p' and the other two are from F_2' and F_3' respectively, to find Ψ' . (c) For other cases that cannot be handled by cases 1 and 2, we need to enumerate all the combinations of three vertices from F_1' , F_2' , and F_3' respectively to compute Ψ' .*

We present CExact+ in Algorithm 4. It first runs CAPAcc and gets the set F_1' of fixed vertices (line 2). Then, it uses the annular contraction to get the tight upper and lower bounds (line 3). Next, it computes the optimal solution Ψ' considering three different cases (line 4 for case (a), lines 5-6 for case (b), and lines 7-8 for case (c)) in Lemma 17. The additional variables are updated (line 9). Finally, the SAC Ψ' is returned (line 10).

6 EXPERIMENTAL RESULTS

6.1 Setup

Datasets. We consider four real datasets: Brightkite,³ Gowalla³, Flickr⁴ and Foursquare.⁵ For all the datasets, each vertex represents a user and each link represents the friendship between two users. Both Brightkite dataset and Gowalla dataset contain a collection of check-in data shared by users of Brightkite service and Gowalla service. In particular, for Brightkite dataset, there are 4,491,143 checkins collected during the period of Apr. 2008 - Oct. 2010 on 772,783 distinct places; the Gowalla dataset contain 6,442,892 checkins collected on 1,280,969 places. In Flickr dataset, we mark the user a location if she has taken a photo there. The Foursquare dataset [3] is extracted from Foursquare website. Each user has a location of her hometown position. It contains 33,278,683 checkin records collected during a period from April 2012 to September 2013. For Brightkite, Gowalla, and Foursquare datasets, we consider the users' locations can be both static (for SAC search) and dynamic (for CSAC search). The static location associated with a user is the place she checks in (or takes photos) most frequently, or hometown location. Note that users without locations are skipped.

TABLE 5
Parameter Settings

Parameter	Range	Default
k	4, 7, 10, 13, 16	4
ϵ_F (AppFast)	0.0, 0.5, 1.0, 1.5, 2.0	0.5
ϵ_A (AppAcc)	0.01, 0.05, 0.1, 0.5, 0.9	0.5
θ	10^{-6} , 10^{-5} , 10^{-4} , 10^{-3} , 10^{-2}	10^{-4}
n	20%, 40%, 60%, 80%, 100%	100%

We have also performed experiments on synthetic datasets. We are not aware of any existing spatial graph data generators. Thus, we create synthetic data in the following way. First, we use GTGraph,⁶ a well-known graph generator, to generate a (non-spatial) graph. We adopt the default parameter values of GTGraph. The degrees of the graph follow a power-law distribution, which is often exhibited in social networks. To generate the location of each graph vertex, we first randomly select a vertex v and give it a random position in the $[0, 1] \times [0, 1]$ space. Then we place v 's neighbors at random positions, whose distances follow a normal distribution with mean μ and standard deviation σ . We repeat this step for other vertices, starting from v 's neighbors, until every vertex has a location. We set $\mu = 0.09$ and $\sigma = 0.16$; these values are derived from the Brightkite dataset. Following these settings, we create two spatial graphs, namely Syn1 and Syn2.

The statistics of each dataset are summarized in Table 4, where the average degree ($\hat{d}$), the average betweenness centrality (BC), and the clustering coefficient (CC) of each vertex in the dataset are included. Without loss of generality, we normalize all the locations of each dataset into the unit square $[0, 1]^2$.

Parameters. We consider 5 parameters: k (denoting the minimum degree), ϵ_F (the parameter of AppFast), ϵ_A (the parameter of AppAcc), θ (the parameter of θ -SAC search), and the percentage of vertices n . In general, the value of k should not be too small or too large, because a small value of k (e.g., $k = 1$ or 2) may result in a community with many vertices having just few neighbors; a too large value of k ($k = 20$) may result in many "false queries (i.e., queries returning empty results). The ranges of the parameters and their default values are shown in Table 5. The default values of ϵ_F and ϵ_A are set as 0.5, since these values practically result in good approximation ratios with reasonable efficiency. Note that when varying n for scalability testing, we randomly extract subgraphs of 20, 40, 60, 80 and 100 percent vertices of the original graph with a default value of 100 percent. When varying a certain parameter, the values for all the other parameters are set to their default values.

Queries of SAC Search. For each dataset, we randomly select 200 query vertices with core numbers of 4 or more. Such a core number constraint ensures a meaningful community containing the query vertex. In the results reported in the following, each data point is the average result for these 200 queries. We use the term "AppFast (ϵ)" ("AppAcc (ϵ)") to denote the algorithm AppFast (AppAcc) with the parameter $\epsilon_F = \epsilon$ ($\epsilon_A = \epsilon$).

Queries of CSAC Search. We use three real datasets, i.e., Brightkite, Gowalla, and Foursquare, to evaluate the efficiency of CSAC search. In each of these datasets, users have

3. <http://snap.stanford.edu/data/index.html>

4. <https://www.flickr.com/>

5. https://archive.org/details/201309_foursquare_dataset_umh

6. <http://www.cse.psu.edu/~madduri/software/GTgraph/>

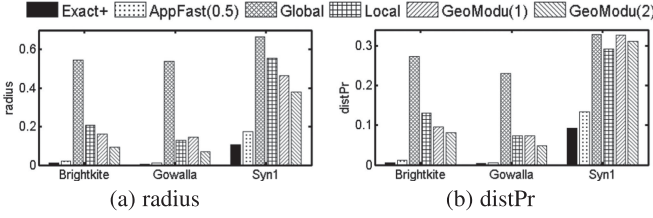


Fig. 12. Comparison with existing CD and CS methods.

continuous check-in records. We first sort all the checkin records in chronological order, and then we divide them into two groups G_1 and G_2 , where G_2 contains the checkin records in the last 30 days while G_1 contains the other records. In the experiments, we first update users' locations by records in G_1 , and then simulate users' movement by records from G_2 . For each proposed algorithm (CAppFast, CAppAcc, and CExact+), we design its counterpart by simply calling the corresponding SAC search algorithm repeatedly without any optimization. The parameter settings are the same as that of SAC search. In the results reported, each data point is the average result for these queries.

We implement all the algorithms in Java, and run experiments on a machine having a quad-core Intel i7-3770 3.40 GHz processor and 32 GB of memory, with Ubuntu installed.

6.2 Effectiveness Evaluation

6.2.1 Comparison with the State-of-the-Arts

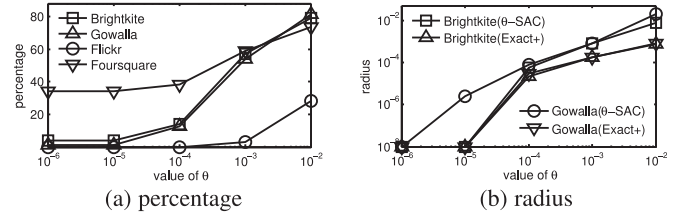
In this section, we show that SAC search returns communities with higher spatial cohesiveness compared with the state-of-the-art CR methods: Global [10], Local [11] and GeoModu [18]. The first two methods are CS methods designed for non-spatial graphs, while GeoModu is a CD method for spatial graphs. We also compare it with θ -SAC search. We briefly introduce these algorithms as follows (Let q be a query vertex):

- **Global**: it finds the k -core containing q .
- **Local**: it expands and explores from q , until it forms a subgraph whose minimum vertex degree is at least k .
- **GeoModu**: it first redefines the weight of each edge of graph G as $e_{i,j} = \frac{1}{d_{i,j}^\mu}$, where $d_{i,j} = |v_i, v_j|$ and μ (1 or 2) is a decay factor, and then detects the communities using modularity maximization. Given a query vertex, we return the community which contains it.
- **θ -SAC search**: it first performs BFS search on G starting at q to find a set S of vertices, which are connected with q and in the circle $O(q, \theta)$, and then returns the k -core containing q in $G[S]$.

Both Global and Local use the minimum degree metric for structure cohesiveness. GeoModu has two variants, i.e., GeoModu (1) and GeoModu (2), as the typical values of μ are 1 and 2. To measure the spatial cohesiveness of a community G_q with MCC $O(c, r)$, we introduce two metrics as follows:

- **radius**: the value of radius r .
- **distPr**: average pairwise distance of vertices of G_q .

Intuitively, lower values of these metrics for a community imply that it achieves higher spatial cohesiveness. To compare these methods, we consider both exact and approximation algorithms. We search communities using these algorithms and compute the average values of above metrics for these communities. We report the results on Brightkite and Gowalla datasets in Fig. 12.

Fig. 13. Results of θ -SAC search.

1. **CS Comparison**. We see that Local performs better than Global, as it finds communities through local expansion. The vertices of communities returned by Global and Local spread in larger areas than those of SAC search methods. For example, the average radii of the MCCs covering the communities of Global and Local are respectively 50 and 20 times larger than that of our approach. The main reason is that they overlook the spatial locations. Note that although G_q is in the smallest MCC, G_q may not be the subgraph with the minimum number of vertices satisfying the minimum degree metric. In other words, a proper subset of vertices in G_q may form a qualified community, which has the same MCC as that of SAC search. Among the SAC search methods, the exact algorithm Exact+ achieves better spatial cohesiveness than approximation algorithms consistently.

2. **CD Comparison**. Since GeoModu considers both links and locations, the returned communities achieve better spatial cohesiveness than Global and Local. However, the average radius and distPr values of GeoModu are larger than SACs because a non-trivial number of queries in GeoModu return communities whose MCCs are smaller than those of SAC (e.g., 19 and 18 percent of queries whose communities returned by GeoModu (1) are in MCCs with smaller radii than those of Exact+, in Brightkite and Gowalla datasets respectively). However, the structure cohesiveness of communities detected by GeoModu is weaker. For example, the corresponding average degrees of vertices in communities returned by GeoModu (1) and GeoModu (2) on Brightkite dataset are 2.2 and 1.1. This is because GeoModu partitions the graph into clusters using a global criterion, i.e., GeoModularity [18], with no reference to the query vertices.

3. **Comparison with θ -SAC Search**. We vary the value of θ in θ -SAC search, and compute the percentage of queries returning non-empty subgraphs. Fig. 13a reports the results. Notice that the percentage is low when θ is small. This is because many users' SACs are spread in large areas. Also, the percentage varies greatly for different datasets. Thus, setting a proper value of θ is not easy. In contrast, SAC search does require the specification of θ , and it always returns an SAC, if there is any. For queries returning non-empty SACs, we compute the average radius of MCCs covering these SACs. We also compute the average radius for MCCs of SACs found by Exact+. Fig. 13b compares their results. We observe that the average radius of MCCs covering SACs found by θ -SAC search is 5 to 10 times larger than that of Exact+. This means that SAC search achieves better spatial cohesiveness than this variant. Hence, SAC search is easier to be used, and also achieves higher spatial cohesiveness than θ -SAC.

6.2.2 Adaptability to Location Changes

To study the adaptability of location changes of CSAC search, we use a "dynamic" spatial graphs, where vertices' locations change frequently. We consider Brightkite dataset and assume the link relationships do not change. We first

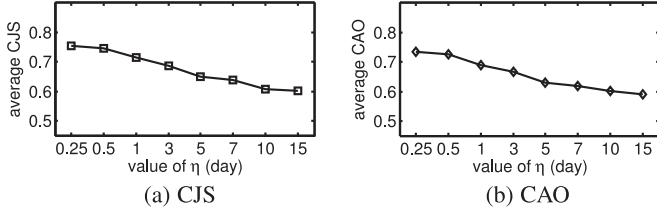


Fig. 14. Effectiveness on dynamic spatial graph.

sort all the checkin records in chronological order. Then, we divide them into two groups R_1 and R_2 , where R_1 contains records collected before 2010 and R_2 contains the remaining records. Finally, we compute the total travel distance of each user, by adding up the distances between each consecutive pair of checkins, and select a set Q of 100 query users, who travel the longest and have at least 20 friends.

To perform queries, we first go through checkin records in R_1 and update users' locations according to their latest checkin timestamps. Then, for each user $q \in Q$, we do the same operation for records in R_2 , and if the record was generated by q , we search her SAC by CExact+. Finally, we get a list of SACs, $L_q = \{C_1, C_2, \dots, C_l\}$, where $C_i (1 \leq i \leq l)$ is an SAC at the timestamp of the i th checkin record, and l is the number of q 's check-in records.

To measure the overlap of member sets and spatial areas between two communities C_i and C_j , we define two metrics: *community jaccard similarity* (CJS) and *community area overlapping* (CAO), based on the classical Jaccard similarity.

$$CJS(C_i, C_j) = \frac{V(C_i) \cap V(C_j)}{V(C_i) \cup V(C_j)}, \quad (9)$$

$$CAO(C_i, C_j) = \frac{A(C_i) \cap A(C_j)}{A(C_i) \cup A(C_j)}, \quad (10)$$

where $V(C_i)$ is the member set of C_i and $A(C_i)$ is the area of the MCC covering C_i . Notice that $CJS(C_i, C_j)$ and $CAO(C_i, C_j)$ range from 0 to 1. A smaller value of CJS (CAO) implies lower overlapping of community member sets (spatial areas).

To show how the values of CJS and CAO vary with time, we select communities from L_{q_i} where the time gap between each pair of communities is at least η , and compute their CJS and CAO values. We report their average results in Fig. 14, where η varies from 0.25 day to 15 days. From Fig. 14a, we observe that, the CJS decreases as the time threshold increases. For example, after 6 hours, the value of CJS decreases to 75 percent, which implies the SACs indeed have changed, but they share a remarkable proportion of common members. Fig. 14b shows that the SACs' locations change frequently over time as well. In addition, we plot the SACs of two mobile users in Fig. 2. Thus, these results well confirm that, CSAC search has high adaptability to location changes.

6.3 Efficiency Evaluation of SAC Search

We report the efficiency results of SAC search on three real datasets in Fig. 15. The results on other datasets are in [43].

1. *Effect of k for Approximation Algorithms.* We show the results in Figs. 15a, 15b, and 15c. AppFast runs consistently faster than AppInc and AppAcc. For example, for the largest real dataset Foursquare, AppFast (0.0) is at least two orders of magnitude faster than AppInc, although they return the same SACs. AppFast (0.0) is 2 to 5 times faster

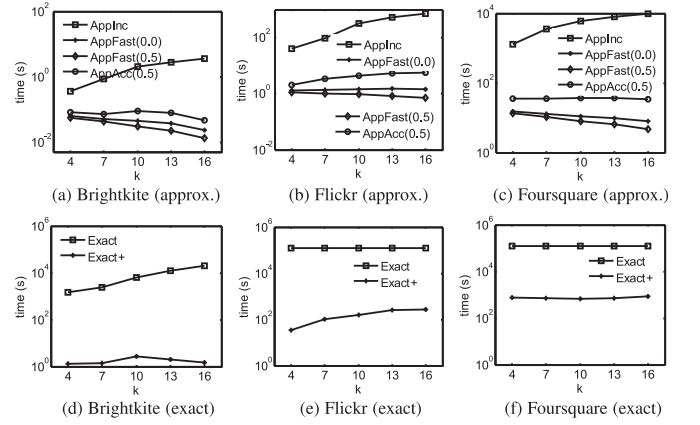
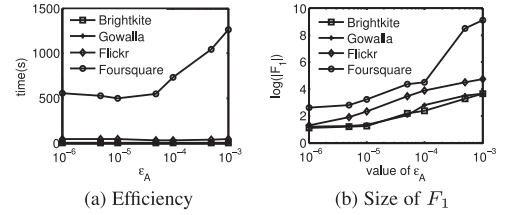


Fig. 15. Efficiency of SAC search on real datasets.

Fig. 16. Effect of ϵ_A on Exact+.

than AppAcc (0.5). This is because AppFast has a lower time complexity. In addition, the running time of AppFast decreases as the value of k increases. This is because $O(q, \delta)$ becomes larger as k increases, and finding a larger $O(q, \delta)$ tends to need less binary search.

The running time of AppInc increases clearly as the value of k grows, because for a larger value of k , the corresponding $O(q, \delta)$ is also larger. As it finds $O(q, \delta)$ starting from the query vertex q incrementally, a larger value of k results in a higher cost.

AppAcc (0.5) is slower than AppFast. This is because its first step is to run AppFast and it needs extra effort to find smaller MCCs. Also, its time cost tends to be stable. As discussed before, the number of anchor points is mainly affected by ϵ_A . Since ϵ_A is always 0.5 for different k , the numbers of anchor points are the same, and thus the running time remains stable.

2. *Effect of k for Exact Algorithms.* Fig. 16a shows that the efficiency of Exact+ is not very sensitive to ϵ_A on all real datasets except Foursquare. Fig. 16b shows $|F_1|$ increases with ϵ_A ; that is, fewer vertices are pruned as ϵ_A grows. Recall in Section 5.5 that Exact+ is composed of two phases. When ϵ_A is small, the cost of Phase (1) dominates the overall cost; as ϵ_A increases, the cost of Phase (2) grows. Thus, there is a local minimum in Fig. 16a. In practice, a sensitivity test can be done to choose a suitable value of ϵ_A .

Figs. 15d, 15e, and 15f show the efficiency results of exact algorithms ($\epsilon_A = 10^{-4}$). We skip the results of Exact, if a single query takes more than 10 hours. Clearly, Exact performs extremely slow, even on the smallest dataset, which calls for the design of more efficient algorithms. Exact+ is at least four orders of magnitude faster than Exact. This is because, it uses AppAcc to find narrow annular regions, in which fixed vertices are supposed to be contained, and thus the number of fixed vertices needed to be enumerated is reduced significantly. The performance of Exact+ either slightly increases

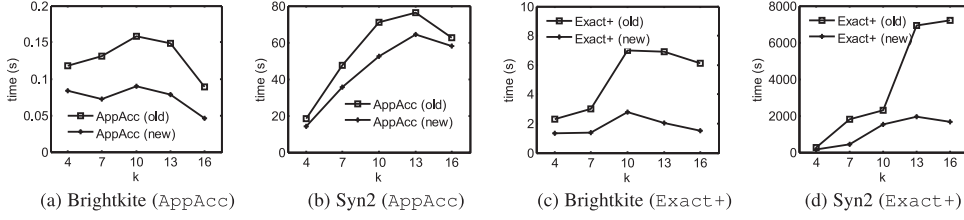


Fig. 17. Improvement of AppAcc and Exact+.

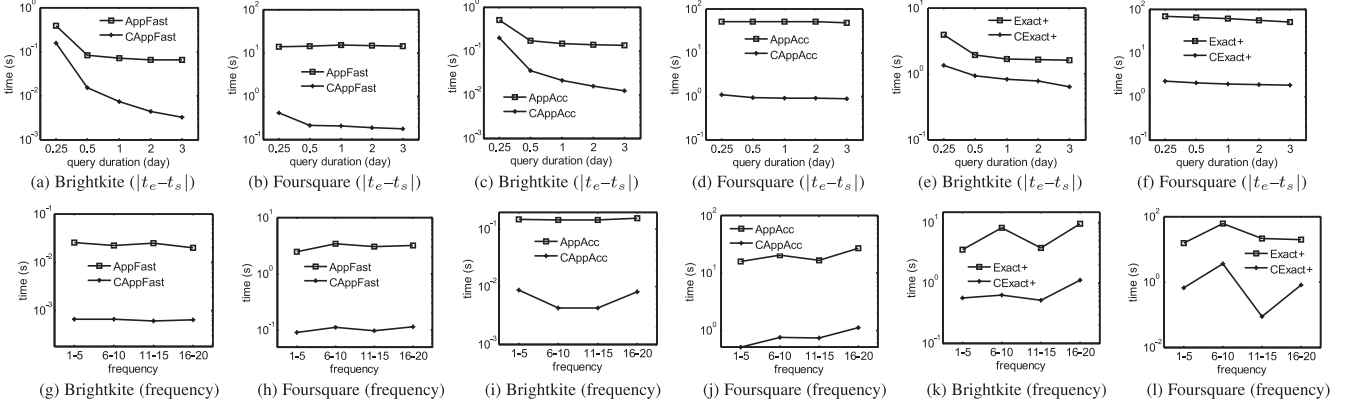


Fig. 18. Efficiency of CSAC search.

or decreases as the value of k increases. This is because, after pruning by the annular regions, the numbers of fixed vertices left may be different, but generally larger datasets have more fixed vertices, and thus more time cost is needed. Additionally, since Exact+ runs AppAcc in the first step, it is slower than AppAcc, but it takes reasonable time, i.e., few seconds, on moderate-size graphs like Brightkite and Syn1.

3. *Improvement of AppAcc and Exact+.* In this experiment, we consider the Brightkite and Syn2 datasets. We vary the value of k , and compare the efficiency of the improved AppAcc and Exact+ with their old versions published in [45]. The results are reported in Figs. 17a, 17b, 17c, and 17d. We observe that, AppAcc and Exact+ are faster than their old versions, i.e., achieving an acceleration up to 2 to 5 times. In addition, their performance trends are similar with those discussed before.

4. *Results on Noisy Graphs.* We consider two new datasets with anomalous vertices and also add different kinds of noise vertices into the datasets of Table 4. The efficiency results on these datasets show that our algorithms generally are very robust to noise vertices, and their parameters are not very sensitive to the noise. In addition, we have evaluated the effect of vertices' betweenness centrality and clustering coefficient on efficiency. Due to the space limitation, we present the detailed experimental results and discussions in the supplemental materials, which can be found on the Computer Society Digital Library at <http://doi.ieeecomputersociety.org/10.1109/TKDE.2018.2845414>.

6.4 Evaluation of CSAC Search

In this section, we study the efficiency of CSAC search and report results in Fig. 18 (results on Gowalla dataset are in [43]).

1. *Effect of Query Duration $|t_e - t_s|$.* In this experiment, we randomly select 100 query users, and set the query duration as 0.25, 0.5, 1.0, 2.0, and 3 days respectively. Then, we run CAlgo and Algo to answer the queries of CSAC search. The efficiency results are reported in Figs. 18a, 18b, 18c, 18d, 18e, and 18f. The exact algorithm CExact+ is slower than

the approximation algorithms, and CAppFast is faster than CAppAcc. We observe that, when the value of query duration increases, the average time of both AppFast and CAppFast becomes smaller, but the decrease of CAppFast is more pronounced. The reason is that, as the query duration increases, CAppFast makes full use of the previous results and does not require repeated computation. Also, with the value of $|t_e - t_s|$ increased, the gap between CAppFast and CAppAcc grows, and CAppFast is up to 80 times faster than CAppAcc. Thus, CAppFast is more suitable for dynamic spatial graphs.

Besides, CAppAcc is over an order of magnitude faster than AppAcc, and the gap between them also increases as the length of query duration grows and their performance trends are similar with CAppFast, as discussed before. In addition, CExact+ is faster than Exact on dynamic graphs. The performance trends are also similar with those of CAppFast and CAppAcc.

2. *Effect of Update Frequency.* In this experiment, we first count the numbers of check-in times for each user. Then, we divide users into four groups according to their check-in frequency, each corresponding to 1~5, 6~10, 11~15, and 16~20 times. Next, for each group we randomly select 100 users. Finally, we perform CSAC search for users in each group, and report the results in Figs. 18g, 18h, 18i, 18j, 18k, and 18l. We observe that, when the value of frequency increases, the efficiency of AppFast and CAppFast does not change much, so both of them are not sensitive to the frequencies of users' movement. Again, CAppFast is still much more than AppFast, which is consistent with the previous results.

Besides, the performance of CAppAcc is not very stable when varying the value of frequency. This is because, if the query user q does not move, CAppAcc needs little computational cost to update the SAC; otherwise, CAppAcc needs to consider more anchor points, which involves more cost. Again, the gap between AppAcc and CAppAcc is over an order of magnitude. In addition, the results of CExact+ are similar with those of CAppAcc.

7 CONCLUSIONS

In this paper, we examine the online search of communities that exhibit both structure and spatial cohesiveness on spatial graphs. Given a vertex q of a graph G , we study the SAC search problem, which finds the community containing q within the smallest minimum covering circle. We propose two exact algorithms, and three efficient approximation algorithms. We further study the problem of continuous SAC (CSAC) search on dynamic spatial graphs, and propose three efficient algorithms. We evaluate the algorithms on both real and synthetic datasets, and find that SAC search achieves better effectiveness than the existing methods. Also, the approximation algorithms are very fast on large datasets. In addition, the algorithms of CSAC search are very efficient.

In the future, we will examine other structure cohesiveness measures (e.g., k -truss and k -clique) and spatial cohesiveness measures (e.g., pair-wise vertex distances). We will study how to adapt our SAC search solutions for directed spatial graphs using D-core [46], which is extended from k -core for directed graphs.

ACKNOWLEDGMENTS

Reynold Cheng, Yixiang Fang, Xiaodong Li, and Jiafeng Hu were supported by the Research Grants Council of Hong Kong (RGC Projects HKU 17229116 and 17205115) and HKU (Projects 104004572, 102009508, 104004129, 104005000). Xiaojun Chen was supported by NSFC under Grant no. 61773268.

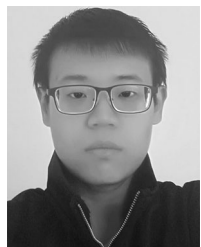
REFERENCES

- [1] N. Armenatzoglou, et al., "A general framework for geo-social query processing," *Proc. VLDB Endowment*, vol. 6, no. 10, pp. 913–924, 2013.
- [2] W. Zhang, et al., "Combining latent factor model with location features for event-based group recommendation," in *Proc. 19th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2013, pp. 910–918.
- [3] M. Sarwat, et al., "LARS*: A scalable and efficient location-aware recommender system," *IEEE Trans. Knowl. Data Eng.*, vol. 26, no. 6, pp. 1384–1399, Jun. 2014.
- [4] Y. Fang, R. Cheng, W. Tang, S. Maniu, and X. Yang, "Scalable algorithms for nearest-neighbor joins on big trajectory data," *IEEE Trans. Knowl. Data Eng.*, vol. 28, no. 3, pp. 785–800, Mar. 2016.
- [5] K. M. MacQueen, et al., "What is community? An evidence-based definition for participatory public health," *Amer. J. Public Health*, vol. 91, no. 12, pp. 1929–1938, 2001.
- [6] B. R. S. Saurabh Sen, Anshuman, *International Perspectives on Socio-Economic Development in the Era of Globalization*. Hershey, PA, USA: IGI Global, 2016.
- [7] R. E. Ricklefs, "Community diversity: Relative roles of local and regional processes," *Sci. (Washington)*, vol. 235, no. 4785, pp. 167–171, 1987.
- [8] M. E. Newman and M. Girvan, "Finding and evaluating community structure in networks," *Phys. Rev. E*, vol. 69, no. 2, pp. 026–113, 2004.
- [9] S. Fortunato, "Community detection in graphs," *Phys. Rep.*, vol. 486, no. 3, pp. 75–174, 2010.
- [10] M. Sozio and A. Gionis, "The community-search problem and how to plan a successful cocktail party," in *Proc. 16th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2010, pp. 939–948.
- [11] W. Cui, Y. Xiao, H. Wang, and W. Wang, "Local search of communities in large graphs," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2014, pp. 991–1002.
- [12] W. Cui, Y. Xiao, H. Wang, Y. Lu, and W. Wang, "Online search of overlapping communities," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2013, pp. 277–288.
- [13] R.-H. Li, L. Qin, J. X. Yu, and R. Mao, "Influential community search in large networks," *Proc. VLDB Endowment*, vol. 8, no. 5, pp. 509–520, 2015.
- [14] X. Huang, L. V. S. Lakshmanan, J. X. Yu, and H. Cheng, "Approximate closest community search in networks," *Proc. VLDB Endowment*, vol. 9, pp. 276–287, 2015.
- [15] Y. Fang, R. Cheng, S. Luo, and H. Jiafeng, "Effective community search for large attributed graphs," *Proc. VLDB Endowment*, vol. 9, no. 12, pp. 1233–1244, 2016.
- [16] D. Guo, "Regionalization with dynamically constrained agglomerative clustering and partitioning," *Int. J. Geographical Inf. Sci.*, vol. 22, no. 7, pp. 801–823, 2008.
- [17] P. Expert, et al., "Uncovering space-independent communities in spatial networks," *Proc. Nat. Acad. Sci. United States America*, vol. 108, no. 19, pp. 7663–7668, 2011.
- [18] Y. Chen, J. Xu, and M. Xu, "Finding community structure in spatially constrained complex networks," *Int. J. Geographical Inf. Sci.*, vol. 29, no. 6, pp. 889–911, 2015.
- [19] M. Barthélemy, "Spatial networks," *Phys. Rep.*, vol. 499, no. 1, pp. 1–101, 2011.
- [20] A. Lancichinetti and S. Fortunato, "Limits of modularity maximization in community detection," *Phys. Rev. E*, vol. 84, no. 6, 2011, Art. no. 066122.
- [21] P. Manchanda, et al., "Social dollars: The economic impact of customer participation in a firm-sponsored online customer community," *Marketing Sci.*, vol. 34, no. 3, pp. 367–387, 2015.
- [22] Y. Chon, N. D. Lane, F. Li, H. Cha, and F. Zhao, "Automatically characterizing places with opportunistic crowdsensing using smartphones," in *Proc. ACM Conf. Ubiquitous Comput.*, 2012, pp. 481–490.
- [23] J. Scott, *Social Network Analysis*. Newbury Park, CA, USA: Sage, 2012.
- [24] M. E. Newman, "Modularity and community structure in networks," *Proc. Nat. Acad. Sci. United States America*, vol. 103, no. 23, pp. 8577–8582, 2006.
- [25] P. Chopade and J. Zhan, "A framework for community detection in large networks using game-theoretic modeling," *IEEE Trans. Big Data*, vol. 3, no. 3, pp. 276–288, Sep. 2017.
- [26] P. Chopade and J. Zhan, "Structural and functional analytics for community detection in large-scale complex networks," *J. Big Data*, vol. 2, no. 1, 2015, Art. no. 11.
- [27] M. Girvan and M. E. Newman, "Community structure in social and biological networks," *Proc. Nat. Acad. Sci. United States America*, vol. 99, no. 12, pp. 7821–7826, 2002.
- [28] Y. Fang, R. Cheng, S. Luo, H. Jiafeng, and H. Kai, "C-explorer: Browsing communities in large graphs," *Proc. VLDB Endowment*, vol. 10, no. 12, pp. 1885–1888, 2017.
- [29] J. Hu, et al., "Querying minimal steiner maximum-connected subgraphs in large graphs," in *Proc. 25th ACM Int. Conf. Inf. Knowl. Manage.*, 2016, pp. 1241–1250.
- [30] S. Babu and J. Widom, "Continuous queries over data streams," *ACM SIGMOD Rec.*, vol. 30, no. 3, pp. 109–120, 2001.
- [31] G. S. Iwerks, et al., "Continuous k-nearest neighbor queries for continuously moving points with updates," in *Proc. 29th Int. Conf. Very Large Data Bases*, 2003, pp. 512–523.
- [32] D. Wu, et al., "Efficient continuously moving top-k spatial keyword query processing," in *Proc. IEEE 27th Int. Conf. Data Eng.*, 2011, pp. 541–552.
- [33] Y. Tao and D. Papadias, "Spatial queries in dynamic environments," *ACM Trans. Database Syst.*, vol. 28, no. 2, pp. 101–139, 2003.
- [34] M. F. Mokbel, et al., "SINA: Scalable incremental processing of continuous queries in spatio-temporal databases," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2004, pp. 623–634.
- [35] A. E. Sarıyüce, et al., "Incremental k-core decomposition: Algorithms and evaluation," *VLDB J.*, vol. 25, no. 3, pp. 425–447, 2016.
- [36] R.-H. Li, J. X. Yu, and R. Mao, "Efficient core maintenance in large dynamic graphs," *IEEE Trans. Knowl. Data Eng.*, vol. 26, no. 10, pp. 2453–2465, Oct. 2014.
- [37] S. B. Seidman, "Network structure and minimum degree," *Social Netw.*, vol. 5, no. 3, pp. 269–287, 1983.
- [38] V. Batagelj and M. Zaversnik, "An o(m) algorithm for cores decomposition of networks," *CoRR*, vol. cs.DS/0310049, 2003, <http://arxiv.org/abs/cs.DS/0310049>
- [39] D. J. Elzinga and D. W. Hearn, "The minimum covering sphere problem," *Manage. Sci.*, vol. 19, no. 1, pp. 96–104, 1972.
- [40] J. Elzinga, et al., "Geometrical solutions for some minimax location problems," *Transp. Sci.*, vol. 6, no. 4, pp. 379–394, 1972.
- [41] N. Megiddo, "Linear-time algorithms for linear programming in R3 and related problems," in *Proc. 23rd Annu. Symp. Found. Comput. Sci.*, 1982, pp. 329–338.

- [42] T. Guo, X. Cao, and G. Cong, "Efficient algorithms for answering the m-closest keywords query," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2015, pp. 405–418.
- [43] Y. Fang, et al., "On spatial-aware community search (technical report)," (2018). [Online]. Available: <http://www.cs.hku.hk/research/techreps/document/TR-2016-04.pdf>
- [44] R. A. Finkel and J. L. Bentley, "Quad trees: A data structure for retrieval on composite keys," *Acta Informatica*, vol. 4, no. 1, pp. 1–9, 1974.
- [45] Y. Fang, R. Cheng, X. Li, S. Luo, and J. Hu, "Effective community search over large spatial graphs," *Proc. VLDB Endowment*, vol. 10, no. 6, pp. 709–720, 2017.
- [46] C. Giatsidis, et al., "D-cores: Measuring collaboration of directed graphs based on degeneracy," in *Proc. IEEE 11th Int. Conf. Data Mining*, 2011, pp. 201–210.



Yixiang Fang received the PhD degree from the University of Hong Kong (HKU), in 2017. Currently, he is a postdoc researcher with the University of New South Wales, under the supervision of Prof. Xuemin Lin. His research interests mainly focus on graph queries.



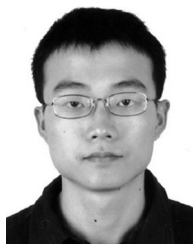
Zheng Wang is working toward the master's degree in the Department of Computer Science, HKU. His research interests focus on graph queries.



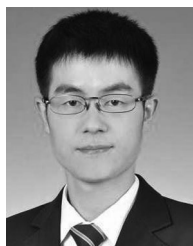
Reynold Cheng is an associate professor with the Department of Computer Science, University of Hong Kong. He has served as a PC member and reviewer for international conferences (e.g., SIGMOD, VLDB, ICDE, and KDD) and journals (e.g., the *IEEE Transactions on Knowledge and Data Engineering*, the *ACM Transactions on Database Systems*, the *VLDB Journal*, and the *Information Systems Journals*). He is an associate editor of the *IEEE Transactions on Knowledge and Data Engineering*, and on the EIC selection committee for the *IEEE Transactions on Knowledge and Data Engineering*. He is a member of the IEEE and ACM.



Xiaodong Li is working toward the PhD degree in the Department of Computer Science, University of Hong Kong. His research area is graph management.



Siqiang Luo is working toward the PhD degree in the Department of Computer Science, University of Hong Kong. His research area is graph management.



Jiafeng Hu is working toward the PhD degree in the Department of Computer Science, University of Hong Kong. His research area is graph management.



Xiaojun Chen is an assistant professor with Shenzhen University. His research interests include data mining and machine learning.

▷ For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/publications/dlib.